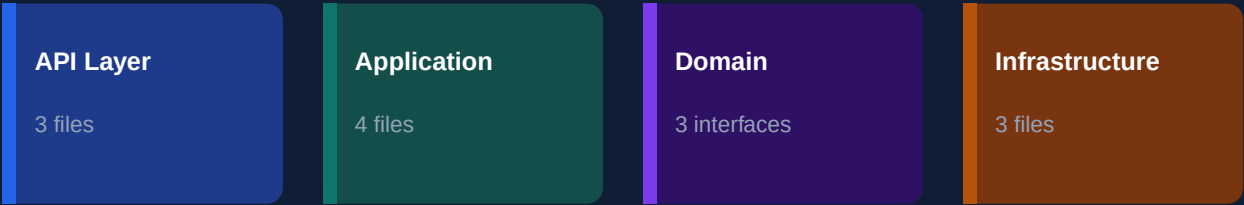


Clean Architecture • 13 Files • 4 Layers

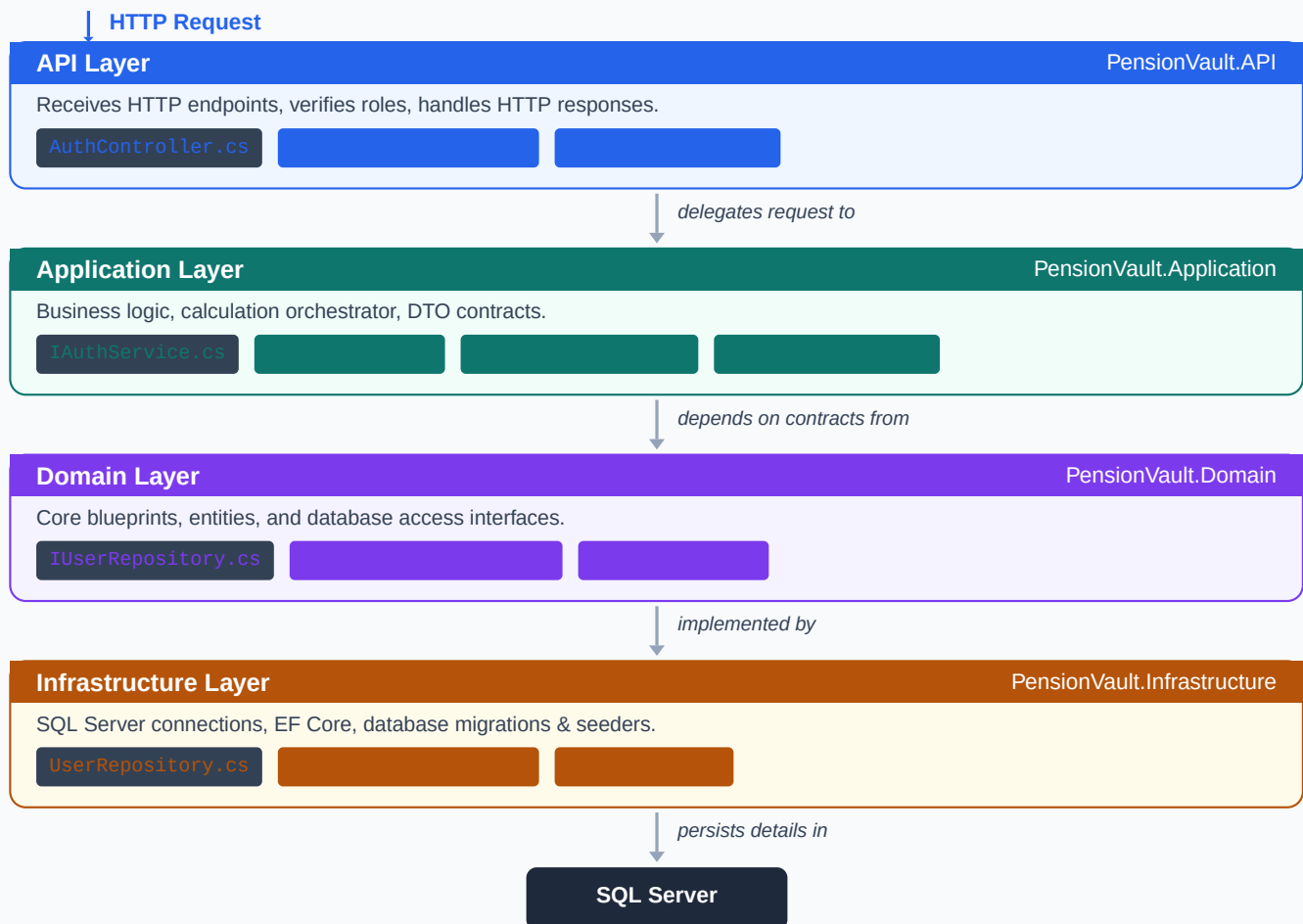
PensionVault

Complete Developer Walkthrough Guide

Module 2.1 — Identity & Access Management
Module 2.7 — Pension Annuity & Settlement

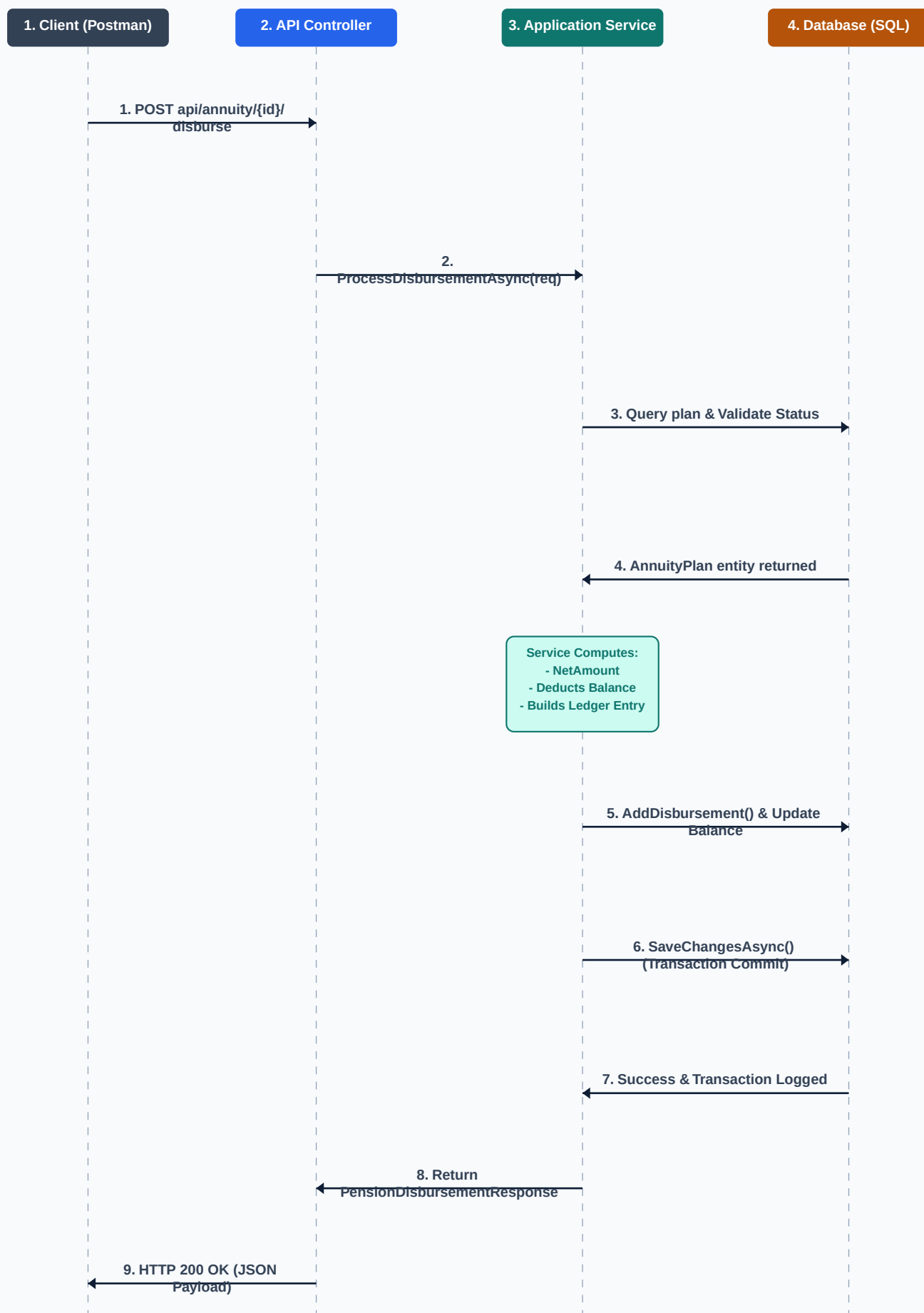


- ASP.NET Core
- EF Core
- JWT Bearer
- BCrypt
- Serilog
- SQL Server



PensionVault — End-to-End Request Payout Flow

Step-by-step execution path of a monthly pension disbursement trigger



1. PensionVault Overview & Architecture Map

1.1 What is PensionVault?

PensionVault is a robust, enterprise-grade pension administration backend built on Clean Architecture principles. It serves as a secure central ledger and core transaction manager to automate member lifecycle stages including:

- Member Enrolment & Schemes Configuration (EPF, Gratuity, Superannuation, NPS, PPF, etc.)
- Contribution Remittance Processing (Employer collections, bank file uploads, and reconciliations)
- Ledger Accounting (Double-entry recording of contributions, interest credits, and claim debits)
- Retirement Claim Disbursements & Automated Annuity Payout Distributions

1.2 Codebase Project Structure Directory Map

Layer Name	Folder Path	Primary Responsibility
API Layer	PensionVault.API/	Defines HTTP Controllers, routing templates, global filters, and Kestrel server startup config.
Application	PensionVault.Application/	Houses service logic interfaces/implementations, validation constraints, and request DTOs.
Domain Layer	PensionVault.Domain/	Central blueprints layer containing SQL entities, custom status enums, and repository abstractions.
Infrastructure	PensionVault.Infrastructure/	Interacts with physical SQL Server using EF Core migrations, database seeds, and raw repositories.

1.3 Architectural Map -- 13 Files / 4 Layers

API Layer	PensionVault.API
Receives HTTP endpoints, verifies roles, handles HTTP responses. AnnuityController.cs AuthController.cs AuditLogFilter.cs	
Application Layer	PensionVault.Application
Houses all business rules, calculations, and DTO data mappings. IAuthService.cs AuthService.cs IAnnuityService.cs AnnuityService.cs	
Domain Layer	PensionVault.Domain
Defines the core interfaces and contracts for data access. IUserRepository.cs IAnnuityRepository.cs IUnitOfWork.cs	
Infrastructure Layer	PensionVault.Infrastructure
Connects to SQL Server using EF Core and executes queries. UserRepository.cs AnnuityRepository.cs UnitOfWork.cs	

2. Identity & Access Management (IAM) Workflow & Classes

PensionVault secures its REST API using JSON Web Tokens (JWT) and BCrypt password encryption. The core authentication logic implements a token rotation model for persistent sessions:

- Registration: User details are passed to AuthService. Registration encrypts the plaintext password using BCrypt.HashPassword before staging the entity.
- Login Gate: Login verifies user emails and compares passwords using BCrypt.Verify. On credentials match, the server generates a JWT containing Claims (UserId, Email, Name, Role).
- Token Validation: Program.cs configures AddJwtBearer options to validate: 1. Token Signature, 2. Expire Lifespan, 3. Issuer/Audience match.

- **Security Gates:** Active endpoint controllers are protected using the `[Authorize(Roles = "...")]` attributes, limiting critical actions like create plan or disburse to FundAdmin or Admin roles.
- **Session Renewal:** `RefreshTokenRequest` validates base64-encoded 64-byte refresh token. Generates a fresh JWT pair to preserve authentication state.

2.1 AuthController.cs

Handles registration, login, and token refresh requests by mapping incoming HTTP requests directly to `IAuthService` methods.

```
[HttpPost("login")]
public async Task<IActionResult> Login([FromBody] LoginRequest request)
{
    var result = await _authService.LoginAsync(request);
    return Ok(result);
}
```

Code Segment	Line-by-Line Technical Explanation
<code>[HttpPost("login")]</code>	Defines an HTTP POST route at "api/auth/login" corresponding to this method.
<code>[FromBody] LoginRequest</code>	Uses C# model binding to deserialize the raw incoming JSON body directly into a C# <code>LoginRequest</code> record.
<code>await _authService.LoginAsync(...)</code>	Invokes the <code>AuthService</code> block asynchronously to verify password hashes and account status.
<code>return Ok(result)</code>	Wraps the <code>AuthResponse</code> (JWT + Refresh token + user metadata) inside a standard HTTP 200 OK wrapper.

2.2 AnnuityController.cs

Exposes routing points to create annuity contracts, trigger monthly disbursements, execute nominee settlements, and terminate plans.

```
[HttpPost("{id:guid}/disburse")]
[Authorize(Roles = "FundAdmin,Admin")]
public async Task<IActionResult> ProcessDisbursement(Guid id, [FromBody] ProcessDisbursementRequest request)
{
    var req = request with { AnnuityId = id };
    return Ok(await _annuityService.ProcessDisbursementAsync(req));
}
```

Code Segment	Line-by-Line Technical Explanation
<code>[HttpPost("{id:guid}/disburse")]</code>	Maps route "api/annuity/{id}/disburse". Restricts path parameter "id" to be a valid 128-bit GUID constraint.
<code>[Authorize(Roles = "...")]</code>	Authorization filter gate. Rejects requests with HTTP 403 Forbidden unless user JWT claim holds role "Admin" or "FundAdmin".
<code>request with { AnnuityId = id }</code>	C# C-Sharp 9.0 record mutator pattern. Creates a copy of the request, forcing the body <code>AnnuityId</code> to match the URL segment.
<code>await _annuityService.Process...</code>	Sends updated request data parameters down to the Application service Layer within an async task.

2.3 AuditLogFilter.cs

Interceptors built on the Decorator Pattern to automatically track POST, PUT, and DELETE DB modifications.

```
var resultContext = await next();

if (resultContext.Exception == null &&
    resultContext.HttpContext.Response.StatusCode >= 200 &&
```

```

    resultContext.HttpContext.Response.StatusCode < 300)
{
    // ... extract claims and log action ...
}

```

Code Segment	Line-by-Line Technical Explanation
<code>await next()</code>	Delegate call. Halts filter execution and awaits the execution of the target controller method to complete first.
<code>resultContext.Exception == null</code>	Validation check. Verifies the controller executed successfully without crashing or throwing unhandled errors.
<code>Response.StatusCode >= 200...</code>	Verifies the response returned an HTTP 2xx Success status code before logging changes to avoid false flags.
<code>HttpContext.User.FindFirst(...)</code>	Queries the HttpContext ClaimsPrincipal to extract the User ID claim injected by JWT validation.

3. Application Layer -- Services & Business Workflows

3.2 AuthService.cs (Login Gate)

Implements login credential validation, BCrypt password hashing, and token generation.

```

public async Task<AuthResponse> LoginAsync(LoginRequest request)
{
    var user = await _userRepo.FindByEmailAsync(request.Email)
        ?? throw new UnauthorizedAccessException("Invalid credentials.");

    if (!BCrypt.Net.BCrypt.Verify(request.Password, user.PasswordHash))
        throw new UnauthorizedAccessException("Invalid credentials.");

    return await GenerateAuthResponseAsync(user);
}

```

Code Segment	Line-by-Line Technical Explanation
<code>FindByEmailAsync(...)</code>	Invokes repository interface. Queries SQL table for matches against input email address.
<code>throw new UnauthorizedAccessException</code>	Security measure. Throws exception if user email is missing. Does not indicate whether email or password failed (prevents enumeration).
<code>BCrypt.Net.BCrypt.Verify(...)</code>	Cryptographically compares plaintext input password with salted hash stored in database. Secure computation.
<code>GenerateAuthResponseAsync(...)</code>	Generates symmetric signature JWT token, issues base64 random refresh token, and saves details to DB.

3.4 AnnuityService.cs (Disbursement Process)

Processes monthly pension payments. Decrements fund accounts, issues ledger logs, and secures transactions.

```

var netAmount = plan.MonthlyPension - request.TaxDeducted;
var disbursement = new MonthlyPensionDisbursement {
    AnnuityId = plan.AnnuityId,
    NetAmount = netAmount,
    Status = PensionDisbursementStatus.Disbursed
};
await _annuityRepo.AddDisbursementAsync(disbursement);
account.TotalBalance -= plan.MonthlyPension;
await _unitOfWork.SaveChangesAsync();

```

Code Segment	Line-by-Line Technical Explanation
<code>plan.MonthlyPension - TaxDeducted</code>	Business calculation. Subtracts tax rate to arrive at final net cash payout.
<code>AddDisbursementAsync(...)</code>	Stages insertion of a disbursement payment receipt record matching the payout month/year.
<code>account.TotalBalance -= ...</code>	Decrements gross amount from retiree's personal fund account balance.
<code>_unitOfWork.SaveChangesAsync()</code>	EF Core transaction commit. Commits disbursement creation, account deduction, and ledger updates atomically.

5. Infrastructure Layer Repositories & SQL Queries

5.2 AnnuityRepository.cs

Executes SQL transactions via Entity Framework Core joins.

```
public Task<AnnuityPlan?> FindByIdAsync(Guid annuityId)
    => _context.AnnuityPlans
        .Include(a => a.Member)
        .FirstOrDefaultAsync(a => a.AnnuityId == annuityId);
```

Code Segment	Line-by-Line Technical Explanation
<code>_context.AnnuityPlans</code>	Points target query against AnnuityPlans database table.
<code>.Include(a => a.Member)</code>	Eager loading command. Forces EF Core to generate an SQL INNER JOIN statement to pull related Member details in one query.
<code>.FirstOrDefaultAsync(...)</code>	SQL command: SELECT TOP(1) with WHERE check on AnnuityId. Executed asynchronously to avoid blocking threads.

5.4 Program.cs Middleware Pipeline

Startup configurations and middleware pipeline registrations:

```
app.UseSerilogRequestLogging();
app.UseMiddleware<ExceptionMiddleware>();
app.UseCors("AllowAll");
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
```

Code Segment	Line-by-Line Technical Explanation
<code>UseSerilogRequestLogging()</code>	Logs endpoint route, status code, response time, and request payload parameters for audit.
<code>UseMiddleware<ExceptionMiddleware>()</code>	Global try-catch interceptor. Intercepts runtime errors and formats clean JSON error responses.
<code>UseAuthentication()</code>	Analyzes HTTP request Authorization header, extracts JWT bearer token, and decrypts/verifies claims.
<code>UseAuthorization()</code>	Executes access role verification (e.g. restricts route to Admin/FundAdmin).

6. Annuity Plan Types and Calculations

Plan Type	Business Definition	Payout / Status Logic
LifeAnnuity	Standard lifetime pension	Pays fixed MonthlyPension until member's death. On death: Status -> Terminated. No nominee refund.
JointAnnuity	Survivor benefits for spouse	Pays for member's lifetime. On death: Status -> Suspended. Spouse/nominee can claim via
TemporaryAnnuity	Term-limited payout	Pays for a fixed number of years set at purchase. After the term ends: Status -> Lapsed. Payouts stop.

Plan Type	Business Definition	Payout / Status Logic
GuaranteedAnnuity	Guaranteed return term	Pays for a minimum term (e.g. 10 years). If member dies early, remaining term value is settled to nominee.

Calculation Formulas

Monthly Disbursement Payout:

```
NetAmount = MonthlyPension (from AnnuityPlan, NOT from request)
            - TaxDeducted (from ProcessDisbursementRequest)

FundAccount.TotalBalance -= GrossAmount (MonthlyPension)

LedgerEntry: EntryType    = AnnuityDebit
              Amount      = MonthlyPension
              ReferenceId  = DisbursementId.ToString()
```

Nominee Settlement on Member Death:

```
SettlementAmount = FundAccount.TotalBalance (current balance)

AnnuityPlan.Status = AnnuityStatus.Settled
AnnuityPlan.NomineeDetails = "{NomineeName} (Settled to {BankAccountRef})"

FundAccount.TotalBalance -= SettlementAmount -- reduces balance to 0.00

LedgerEntry: EntryType    = AnnuityDebit
              ReferenceId  = "SETTLEMENT-{annuityId}"
```